

Assignment - 9

1. The Basic Assertion Goal:

Understand the simplicity of pytest syntax. Write a file named `test_logic.py`. Create a test function `test_math_operations` that uses simple `assert` statements to verify: That 15×3 equals 45 . That the string `"pytest"` is present within the phrase `"Learning pytest is fun"`.

2. Using Fixtures for Setup Goal:

Replace `setUp` and `tearDown` with the modern `@pytest.fixture` approach. Create a pytest fixture named `sample_dict` that returns a dictionary: `{"name": "Alice", "role": "Dev"}`. Write a test function `test_dict_keys` that: Accepts the fixture as an argument. Asserts that the key `"role"` exists in the dictionary. Asserts that the value of `"name"` is `"Alice"`.

3. Handling Exceptions Goal:

Use the `pytest.raises` context manager. Write a function `get_element(my_list, index)` that returns an item from a list. Write a test function `test_index_error` that uses `with pytest.raises(IndexError):` to verify that attempting to access index `10` of the list `[1, 2, 3]` correctly triggers an error.

4. Parameterized Testing Goal:

Run the same test logic with multiple sets of data. Use the `@pytest.mark.parametrize` decorator to create a single test function `test_is_even`. Pass three different inputs to the test: `2`, `10`, and `22`. The test should assert that each input `% 2 == 0`. Observe how pytest treats these as three separate test cases in the output.

5. Cleaning Up with Fixture Yields Goal:

Manage resources (like files or database connections) properly. Create a fixture named `temp_file`. In the Setup phase : Create a new text file named `test.txt` and write `"Hello World"` into it. Yield the filename to the test. In the Teardown phase (after the yield): Use the `os` module to delete the file. Write a test `test_file_content` that reads the file and verifies the text matches.

CODE:

test_logic.py

```
import pytest
import os

# --- 1. The Basic Assertion ---
def test_math_operations():
    # Verify multiplication
    assert 15 * 3 == 45
    # Verify string membership
    assert "pytest" in "Learning pytest is fun"
```

```

# --- 2. Using Fixtures for Setup ---
@pytest.fixture
def sample_dict():
    return {"name": "Alice", "role": "Dev"}

def test_dict_keys(sample_dict):
    assert "role" in sample_dict
    assert sample_dict["name"] == "Alice"

# --- 3. Handling Exceptions ---
def get_element(my_list, index):
    return my_list[index]

def test_index_error():
    with pytest.raises(IndexError):
        get_element([1, 2, 3], 10)

# --- 4. Parameterized Testing ---
@pytest.mark.parametrize("number", [2, 10, 22])
def test_is_even(number):
    assert number % 2 == 0

# --- 5. Cleaning Up with Fixture Yields ---
@pytest.fixture
def temp_file():
    # Setup: Create the file
    filename = "test.txt"
    with open(filename, "w") as f:
        f.write("Hello World")

    yield filename # This is where the test runs

    # Teardown: Delete the file
    if os.path.exists(filename):
        os.remove(filename)

def test_file_content(temp_file):
    with open(temp_file, "r") as f:
        content = f.read()
    assert content == "Hello World"

```

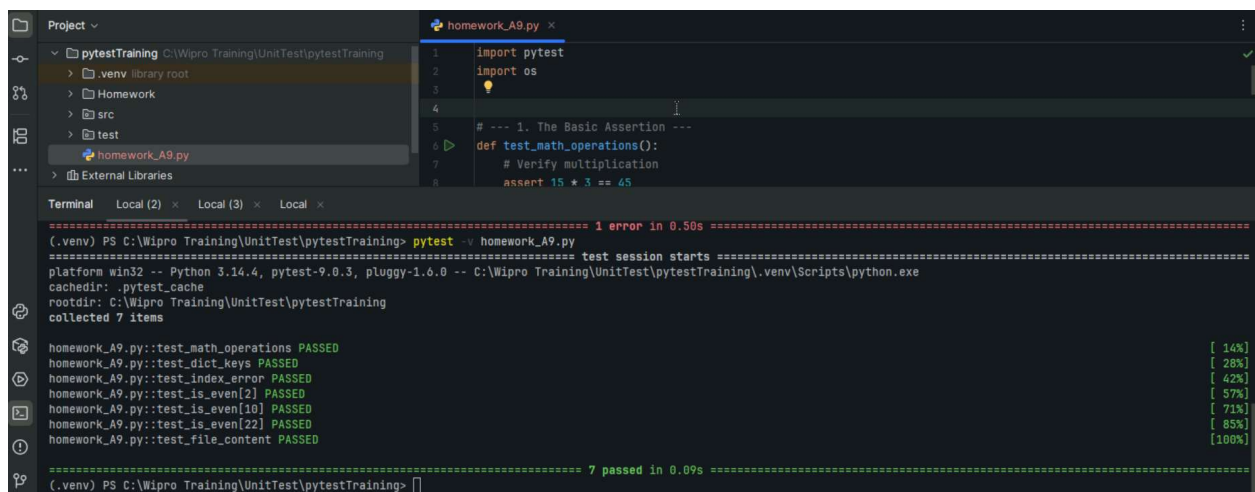
Outputs:

```
(.venv) PS C:\Wipro Training\UnitTest\pytestTraining> pytest -v test_logic.py
```

```
=====
===== test session starts
=====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- C:\Wipro
Training\UnitTest\pytestTraining\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Wipro Training\UnitTest\pytestTraining
collected 7 items

test_logic.py::test_math_operations PASSED
[ 14%]
test_logic.py::test_dict_keys PASSED
[ 28%]
test_logic.py::test_index_error PASSED
[ 42%]
test_logic.py::test_is_even[2] PASSED
[ 57%]
test_logic.py::test_is_even[10] PASSED
[ 71%]
test_logic.py::test_is_even[22] PASSED
[ 85%]
test_logic.py::test_file_content PASSED
[100%]

=====
===== 7 passed in 0.03s
=====
```



The screenshot shows an IDE with a project named 'pytestTraining' and a file named 'homework_A9.py'. The file contains the following code:

```
1 import pytest
2 import os
3
4
5 # --- 1. The Basic Assertion ---
6 def test_math_operations():
7     # Verify multiplication
8     assert 15 * 3 == 45
```

The terminal output shows the results of the test session:

```
(.venv) PS C:\Wipro Training\UnitTest\pytestTraining> pytest -v homework_A9.py
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- C:\Wipro Training\UnitTest\pytestTraining\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Wipro Training\UnitTest\pytestTraining
collected 7 items

homework_A9.py::test_math_operations PASSED [ 14%]
homework_A9.py::test_dict_keys PASSED [ 28%]
homework_A9.py::test_index_error PASSED [ 42%]
homework_A9.py::test_is_even[2] PASSED [ 57%]
homework_A9.py::test_is_even[10] PASSED [ 71%]
homework_A9.py::test_is_even[22] PASSED [ 85%]
homework_A9.py::test_file_content PASSED [100%]

===== 7 passed in 0.09s =====
(.venv) PS C:\Wipro Training\UnitTest\pytestTraining>
```